

BIỂU DIỄN TRI THỨC VÀ TÌM KIẾM NĂNG CAO

Từng bước cài đặt tìm kiếm

DFS, BFS, UCS, Greedy và A* trên Python

Giảng viên: Nguyễn Bích Vân IAI-UET-VNU

Phiên bản: Hướng dẫn thực hành – có lời giải ví dụ mẫu

Đích đến của hướng dẫn: không chỉ bấm Run, mà tự trả lời được: “Biên chứa gì? Lấy nút nào tiếp theo? Khi nào đổi cha? Vì sao dừng?”

Cách học mỗi thuật toán

Dự đoán 3 bước trên giấy → đọc đoạn mã tương ứng → chạy từng bước → đối chiếu → thay đổi một dữ liệu → viết kiểm thử. Ghi dự đoán trước khi mở kết quả; sửa được dự đoán sai cũng là một phần học tập.

Thư viện và thao tác VS Code được đối chiếu tài liệu chính thức ở cuối hướng dẫn. Không cần NumPy, NetworkX hoặc Jupyter; các chương trình trong bộ chỉ dùng thư viện chuẩn.

Phân biệt ba nhóm tệp

Nhóm	Mở để làm gì?
*_demo.py, run_demo.py	Mã mẫu hoàn chỉnh; đọc, chạy và thử thay đổi trên bản sao.
bai_lam.py	Khung nắm hàm sinh viên tự hoàn thiện. NotImplementedError ở đây là thông báo chưa làm bài, không phải lỗi demo.

1. Mở đúng thư mục và chạy lần đầu

1. Giải nén toàn bộ gói. Trong VS Code: **File** → **Open Folder**, chọn thư mục **Greedy_AStar_11Bai** chứa trực tiếp **run_demo.py**. Không chạy tệp bên trong cửa sổ ZIP.
2. Chọn **Python: Select Interpreter** trong Command Palette (Ctrl+Shift+P). Dùng Python đang có trên máy, kể cả Python của Anaconda. Bộ này dùng các cú pháp có từ Python 3.10.
3. Chọn **Terminal** → **New Terminal**. Chạy các dòng bên dưới *lần lượt*, không dán tất cả khi chương trình đang chờ Enter.

```
python --version
python structures_demo.py
python run_demo.py --algo dfs --summary
python run_demo.py --algo dfs --step
```

Không gõ cd python: trong gói mới này, các tệp chạy chính đặt ở thư mục gốc để tránh lỗi thiếu thư mục con. Có thể chạy trực tiếp `python dfs_demo.py -step`; các tệp `common.py` và `data_mau.py` vẫn cần nằm cùng thư mục.

Khi có nhiều bản Python trên Windows

Nút Run của VS Code sử dụng interpreter đã chọn; lệnh `python` trong Terminal phụ thuộc môi trường Terminal. Kiểm tra đường thực thi:

```
python -c "import sys; print(sys.executable)"
```

Không cần cài thêm Anaconda nếu đã có Python phù hợp. Nếu tệp ở thư mục khác, chuyển tới thư mục bằng `cd "đường dẫn"`; dấu ngoặc kép giữ khoảng trắng trong tên.

Hai cách chạy từng bước

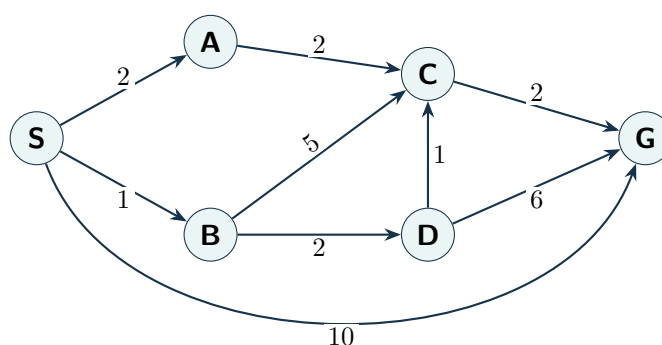
-step: chương trình gọi `input()` ngay *trong quá trình thực thi*, sau INIT hoặc mỗi lần EXPAND/SKIP. Mỗi Enter cho phép tiếp tục vòng lặp tiếp theo; đây không phải phát lại trace tính sẵn. Gói cũ có chế độ phát lại; **gói này dùng dừng trực tiếp theo sự kiện**.

Breakpoint + F10: dừng trước câu lệnh cụ thể trong VS Code, quan sát biến từng dòng. Hai cách khác mức chi tiết, không thay thế nhau. Ctrl+C dừng chế độ Terminal; Shift+F5 dừng trình gỡ lỗi.

2. Mã hóa bài toán trước khi viết tìm kiếm

Một đỉnh là một trạng thái; một cạnh mô tả hành động chuyển trạng thái và phí hành động. `graph[u]` liệt kê các hành động *được phép* từ `u`. Thứ tự list là thứ tự xét hàng xóm; các cặp (v, w) không phải tọa độ, mà là *đỉnh đến và chi phí*.

```
GRAPH = {
  "S": [("A", 2), ("B", 1), ("G", 10)],
  "A": [("C", 2)],
  "B": [("C", 5), ("D", 2)],
  "C": [("G", 2)],
  "D": [("C", 1), ("G", 6)],
  "G": [], "X": []
}
H = {"S": 4, "A": 3, "B": 4, "C": 2,
     "D": 3, "G": 0, "X": 0}
```



X là đỉnh cô lập, dùng để thử trường hợp không tới được. $S \rightarrow A$ không tự tạo $A \rightarrow S$. `graph["G"] = []` là đích không có cạnh ra, không phải dữ liệu bị thiếu.

Ba con số không được trộn

$g(u)$: chi phí đã trả từ S đến `u` trên đường đang xét. $h(u)$: ước lượng chi phí còn lại, do bài toán cung cấp hoặc do ta thiết kế. $f(u) = g(u) + h(u)$: khóa A*. Ví dụ tới A qua S: $g = 2, h = 3, f = 5$. Mặc dù h nhỏ, đường đã đi tới `u` có thể rất đắt.

Bài giảng gọi h là *hàm kinh nghiệm*. Trong mã, `H` là một dict tra giá trị. Đối với mê cung, h được tính bằng công thức Manhattan. Hợp lệ về kiểu dữ liệu không tự chứng minh h chấp nhận được.

3. Ba cấu trúc dữ liệu: nhìn trước, thực hiện, nhìn sau

3.1 Ngăn xếp: vào sau, lấy trước

```
stack = ["B", "A"]
before = stack.copy()
u = stack.pop()
print("before:", before, "pop:", u, "after:", stack)
# before: ['B', 'A'] pop: A after: ['B']
```

Không viết `print(stack, stack.pop())` để mô tả “trước”. Đối số `stack` là tham chiếu tới list; `pop` làm list thay đổi trước khi print in. `before = stack` cũng chỉ tạo bí danh; `copy()` mới chụp độc lập list ở mức này.

3.2 Hàng đợi: vào trước, lấy trước

```
from collections import deque
queue = deque(["A", "B"])
before = list(queue)
u = queue.popleft()
queue.append("C")
print(before, u, list(queue)) # ['A', 'B'] A ['B', 'C']
```

`deque` cho phép lấy đầu và thêm cuối trực tiếp. Dùng `list.pop()` cho BFS sẽ đổi chiến lược thành lấy cuối, không còn FIFO.

3.3 Heap: ưu tiên nhỏ nhất, không phải list đã sắp toàn bộ

```
from heapq import heappush, heappop
heap = []
heappush(heap, (4, "C"))
heappush(heap, (1, "B"))
heappush(heap, (1, "A"))
before = sorted(heap)
priority, u = heappop(heap)
print(before, priority, u) # [(1, 'A'), (1, 'B'), (4, 'C')] 1 A
```

Tuple so từ trái sang phải: ưu tiên trước, tên đỉnh sau. Raw heap không nhất thiết được sắp tăng toàn bộ; dùng `sorted(heap)` chỉ để *xem* theo thứ tự lấy. Không thay heap bằng sort trong mỗi bước thuật toán. Phần in vết thêm chi phí, nên không dùng thời gian có trace để so độ phức tạp chuẩn.

Tự làm ngay

Thêm `(1, "D")` và dự đoán thứ tự lấy hết. Đổi stack thành `["A", "B"]` và dự đoán pop. Trả lời bằng giấy trước rồi chạy `structures_demo.py` hoặc bản sửa riêng.

4. Các biến dùng chung và cách dựng đường

Biến	Hiểu bằng lời
frontier/stack/queue/heap	Các ứng viên đang chờ xét, không phải đường kết quả.
visited/discovered	Đã xử lý/đã phát hiện; thời điểm đánh dấu phụ thuộc biến thể thuật toán.
parent[v]	Cha của v trên đường được giữ; dùng để lần ngược từ đích.
best_g[v]	Chi phí tốt nhất đã biết tới v; chưa luôn là giá tối ưu cuối cùng.
closed	Các trạng thái đã mở rộng; A* mở lại có thể bỏ một đỉnh khỏi đây.
order	Nhật ký đỉnh được lấy ra hợp lệ, không phải chuỗi đường đi.

```

1 def restore_path(parent: dict, goal: str) -> list:
2     path, current = [], goal
3     while current is not None:
4         path.append(current)
5         current = parent[current]
6     path.reverse()
7     return path

```

Ví dụ parent[G]=C, parent[C]=A, parent[A]=S, parent[S]=None. Vòng lặp tạo [G,C,A,S]; reverse thành [S,A,C,G]. None là dấu kết thúc, không dùng làm tên đỉnh.

Trace chụp chính xác cái gì?

Một bản ghi có event, node, before, after, note. INIT khởi tạo; EXPAND đã xử lý hết cạnh ra của u; GOAL lấy được đích, chưa mở rộng đích; SKIP bỏ bản ghi trùng/cũ; FAIL biên hết mà không tìm được đích. Trường before chụp *trước pop*, after chụp *sau cả xử lý bước đó*, không chỉ ngay sau pop.

Các hàm view() trong mã dùng lambda không đổi số để định dạng bản sao hiện tại. Có thể đọc view = lambda: expression như một hàm ngắn def view(): return expression; đây chỉ là phần hỗ trợ quan sát, không phải tiêu chí chọn đỉnh.

Thống kê

expanded: số lần duyệt danh sách kề, không tính đích; với A* có mở lại có thể lớn hơn số trạng thái khác nhau. max_frontier: số *bản ghi thật* trong cấu trúc, kể cả bản ghi cũ; không đồng nhất với toàn bộ bộ nhớ chương trình.

5. Cài đặt DFS

Ý tưởng trước khi đọc mã

Muốn đi sâu vào nhánh đầu tiên: đặt hàng xóm đầu tiên trên đỉnh stack. Vì append thêm cuối và pop lấy cuối, phải đẩy danh sách theo thứ tự đảo. Bản ghi chứa cả cha dự kiến vì một đỉnh chưa xử lý có thể có nhiều bản ghi chờ.

Tự viết theo năm bước

B1 tạo stack ban đầu. B2 chụp before rồi pop, bỏ bản ghi đã thăm. B3 đánh dấu, chốt cha và ghi order, kiểm tra đích. B4 duyệt hàng xóm đảo và đẩy các đỉnh chưa thăm. B5 hết stack thì báo thất bại. Chỉ lúc pop hợp lệ mới quyết định cha cuối của đỉnh trong bản DFS này.

```

1 from common import validate, record, make_result
2
3 def dfs(graph: dict, start: str, goal: str, step: bool = False) -> dict:
4     # B1. Ngăn xếp lưu (đỉnh, cha dự kiến); đỉnh ngăn xếp ở CUỐI list.
5     validate(graph, start, goal)
6     stack = [(start, None)]
7     visited, parent, order, trace = set(), {}, [], []
8     expanded = skipped = 0
9     peak = 1
10    view = lambda: [f"{u}<-{p}" for u, p in reversed(stack)]
11    record(trace, "INIT", start, [], view(), step=step)
12    while stack:
13        # B2. Chụp trước khi pop; không truyền stack và stack.pop vào print.
14        before = view()
15        u, predecessor = stack.pop()
16        if u in visited:
17            skipped += 1
18            record(trace, "SKIP", u, before, view(), "visited", step)
19            continue
20        # B3. Chỉ bản ghi hợp lệ đầu tiên chốt cha và danh dấu đã thăm.
21        visited.add(u)
22        parent[u] = predecessor
23        order.append(u)
24        if u == goal:
25            record(trace, "GOAL", u, before, view(), step=step)
26            return make_result(graph, parent, goal, order, expanded, skipped,
27                               0, peak, trace, True)
28        # B4. Duyệt đảo thu từ hàng xóm đầu tiên được lấy trước.
29        for v, weight in reversed(graph[u]):
30            if v not in visited:
31                stack.append((v, u))
32            expanded += 1
33            peak = max(peak, len(stack))
34            record(trace, "EXPAND", u, before, view(), step=step)
35        # B5. Bilen rong: thất bại hữu hạn, không có đường đến goal.
36        record(trace, "FAIL", goal, [], [], step=step)
37        return make_result(graph, parent, goal, order, expanded, skipped,
38                           0, peak, trace, False)

```

DFS: chạy và đối chiếu ví dụ chung

```
python dfs_demo.py --step
python dfs_demo.py --summary
```

Bước	Xử lý	Biên sau bước; phần tử lấy tiếp ở bên trái
0	Khởi tạo S	$[S^{(\perp)}]$
1	Lấy S	$[A^{(S)}, B^{(S)}, G^{(S)}]$
2	Lấy A	$[C^{(A)}, B^{(S)}, G^{(S)}]$
3	Lấy C	$[G^{(C)}, B^{(S)}, G^{(S)}]$
4	Đích G	$[B^{(S)}, G^{(S)}]$

Đường $S - A - C - G$, giá 6; order S, A, C, G , expanded 3, skipped 0, peak 3.

Sau S , raw stack từ trái sang phải là $[G, B, A]$ (bỏ phần cha cho dễ nhìn); bản xem giảng dạy là $[A, B, G]$. Hai cách mô tả cùng một stack, phần tử pop tiếp đều là A . Tiếp theo A sinh C , C sinh G ; khi pop G hợp lệ, cha $G=C$ chứ không phải S của bản ghi G cũ dưới đáy.

Thử và giải thích

Đổi đích X bằng `python dfs_demo.py -goal X`. Chương trình sẽ xử lý hết vùng tới được và bỏ những bản ghi đã thăm. Trước khi chạy, viết dự đoán ít nhất một bản ghi SKIP. Thêm một khuyên S-S trên bản sao; giải thích tại sao nó không làm vòng lặp chạy mãi.

6. Cài đặt BFS

Ý tưởng trước khi đọc mã

Muốn xét hết tầng hiện tại trước tầng sau: dùng FIFO. Mỗi đỉnh lần đầu phát hiện được đưa vào hàng đợi ngay và được đánh dấu ngay. Nhờ vậy hai đường cùng dẫn tới một đỉnh không tạo hai bản ghi trong hàng đợi.

Tự viết theo ba bước

B1 dùng deque và đặt start vào discovered. B2 popleft, ghi order và kiểm tra đích. B3 duyệt các cạnh ra đúng thứ tự, với đỉnh mới thì đánh dấu, lưu cha và enqueue. Trọng số vẫn có trong dữ liệu nhưng không tham gia lựa chọn FIFO.

```

1 from collections import deque
2 from common import validate, record, make_result
3
4 def bfs(graph: dict, start: str, goal: str, step: bool = False) -> dict:
5     # B1. FIFO: thêm cuối, lấy đầu. Đánh dấu ngay khi đưa vào biên.
6     validate(graph, start, goal)
7     queue = deque([start])
8     discovered, parent = {start}, {start: None}
9     order, trace = [], []
10    expanded, peak = 0, 1
11    record(trace, "INIT", start, [], list(queue), step=step)
12    while queue:
13        # B2. Ban sao before không thay đổi khi hàng đợi bị thay đổi.
14        before = list(queue)
15        u = queue.popleft()
16        order.append(u)
17        if u == goal:
18            record(trace, "GOAL", u, before, list(queue), step=step)
19            return make_result(graph, parent, goal, order, expanded, 0,
20                               0, peak, trace, True)
21        # B3. Một đỉnh chỉ được đưa vào hàng đợi một lần.
22        for v, weight in graph[u]:
23            if v not in discovered:
24                discovered.add(v)
25                parent[v] = u
26                queue.append(v)
27        expanded += 1
28        peak = max(peak, len(queue))
29        record(trace, "EXPAND", u, before, list(queue), step=step)
30    record(trace, "FAIL", goal, [], [], step=step)
31    return make_result(graph, parent, goal, order, expanded, 0,
32                       0, peak, trace, False)

```

BFS: chạy và đối chiếu ví dụ chung

```
python bfs_demo.py --step
python bfs_demo.py --summary
```

Bước	Xử lý	Biên sau bước; phần tử lấy tiếp ở bên trái
0	Khởi tạo S	$[S]$
1	Lấy S	$[A, B, G]$
2	Lấy A	$[B, G, C]$
3	Lấy B	$[G, C, D]$
4	Đích G	$[C, D]$

Đường $S - G$, giá 10; order S, A, B, G , expanded 3, skipped 0, peak 3.

BFS trả cạnh trực tiếp S-G có 1 bước dù phí 10. A và B đã được mở rộng trước khi lấy G, nhưng không nằm trên đường lời giải. G được phát hiện từ S, nên không thay cha G khi một nhánh dài hơn về số cạnh dẫn tới G sau đó.

Thử và giải thích

Đổi riêng trọng số S-G thành 100. Dự đoán order và path của BFS vẫn giữ nguyên, còn cost báo cáo tăng thành 100. Sau đó dùng mê cung thường Bài 8, nơi mỗi bước có phí bằng nhau; đây là điều kiện phù hợp để đường ít cạnh cũng có chi phí nhỏ nhất.

7. Cài đặt UCS

Ý tưởng trước khi đọc mã

Một đỉnh có thể được tìm ra sớm bằng đường đắt rồi sau đó có đường rẻ hơn. Không thể “đánh dấu một lần khi thêm” như BFS rồi cấm mọi cập nhật. Dùng `best_g` để nhớ giá tốt nhất, heap chọn `g` nhỏ nhất.

Tự viết theo ba bước

B1 heap chứa $(0, \text{start})$. B2 pop, so `g` trong bản ghi với `best_g[u]`; nếu khác thì bỏ trước khi ghi order/kiểm tra goal. B3 tính `new_g = g + w`; chỉ khi nhỏ hơn hẳn mới cập nhật đồng thời giá và cha, rồi push một bản ghi mới. Không phải tìm và xóa bản ghi cũ ngay trong heap.

Mã UCS hoàn chỉnh trong ucs_demo.py

```

1 from heapq import heappush, heappop
2 from math import inf
3 from common import validate, record, make_result
4
5 def ucs(graph: dict, start: str, goal: str, step: bool = False) -> dict:
6     # B1. Chỉ phi cung ten dinh quyet dinh thu tu, khong phai thu tu them.
7     validate(graph, start, goal)
8     heap = [(0, start)]
9     best_g, parent = {start: 0}, {start: None}
10    order, trace = [], []
11    expanded = skipped = 0
12    peak = 1
13    view = lambda: [f"{u}(g={g})" for g, u in sorted(heap)]
14    record(trace, "INIT", start, [], view(), step=step)
15    while heap:
16        before = view()
17        g, u = heappop(heap)
18        # B2. Phieu cu phai bi loai TRUOC order va TRUOC kiem tra dich.
19        if g != best_g[u]:
20            skipped += 1
21            record(trace, "SKIP", u, before, view(), f"old g={g}", step)
22            continue
23        order.append(u)
24        if u == goal:
25            record(trace, "GOAL", u, before, view(), f"g={g}", step)
26            return make_result(graph, parent, goal, order, expanded, skipped,
27                               0, peak, trace, True)
28        changes = []
29        # B3. Cai thien chi phi: cap nhat CA best_g lan parent.
30        for v, weight in graph[u]:
31            new_g = g + weight
32            if new_g < best_g.get(v, inf):
33                best_g[v] = new_g
34                parent[v] = u
35                heappush(heap, (new_g, v))
36                changes.append(f"{v}:g={new_g},parent={u}")
37        expanded += 1
38        peak = max(peak, len(heap))
39        record(trace, "EXPAND", u, before, view(), "; ".join(changes), step)
40    record(trace, "FAIL", goal, [], [], step=step)
41    return make_result(graph, parent, goal, order, expanded, skipped,
42                       0, peak, trace, False)

```

UCS: chạy và đối chiếu ví dụ chung

```
python ucs_demo.py --step
python ucs_demo.py --summary
```

Bước	Xử lý	Biên sau bước; phần tử lấy tiếp ở bên trái
0	Khởi tạo S	$[S_0]$
1	Lấy S	$[B_1, A_2, G_{10}]$
2	Lấy B	$[A_2, D_3, C_6, G_{10}]$
3	Lấy A	$[D_3, C_4, C_6, G_{10}]$
4	Lấy D	$[C_4, C_6, G_9, G_{10}]$
5	Lấy C	$[C_6, G_6, G_9, G_{10}]$
6	Bỏ bản ghi C	$[G_6, G_9, G_{10}]$
7	Đích G	$[G_9, G_{10}]$

Đường $S - A - C - G$, giá 6; order S, B, A, D, C, G , expanded 5, skipped 1, peak 4.

B sinh C giá 6, rồi A cải thiện C xuống 4; khi lấy D, đường tới C cũng giá 4 nên cha C vẫn là A. C tạo G giá 6, thay cho G giá 10 từ S và giá 9 qua D. bản ghi C(6) cũ hòa chi phí với G(6), nhưng C đứng trước G theo tên: lấy C cũ ra, bỏ, rồi mới lấy G. Vì vậy skipped=1 nhưng order không lặp C.

Thử và giải thích

Dừng sớm ngay lúc sinh G sẽ sai: giá 10 thay vì 6. Tạo thêm kiểm thử S-G(3), S-A(1), A-G(2) để thấy giá bằng nhau không đổi cha S của G. Dùng F10 tại dòng cập nhật `best_g[v]` để quan sát cả g và parent.

8. Cài đặt GREEDY

Ý tưởng trước khi đọc mã

Chọn ứng viên h nhỏ nhất trong cả biên. Đừng chỉ chọn hàng xóm “trông gần nhất” của đỉnh đang xét: các ứng viên từ những nhánh trước vẫn ở heap. Bản thực hành này giữ cha lần đầu và đánh dấu khi thêm, không tối ưu lại g .

Tự viết theo hai bước

B1 dùng $\text{heap}(h, \text{start})$, discovered và parent . B2 lấy min h ; nếu chưa là đích, thêm tất cả hàng xóm chưa phát hiện với $h[v]$. g không nằm trong khóa ưu tiên. Chi phí đường chỉ được cộng khi báo cáo kết quả.

```

1 from heapq import heappush, heappop
2 from common import validate, record, make_result
3
4 def greedy(graph: dict, start: str, goal: str, h: dict,
5           step: bool = False) -> dict:
6     # B1. Biên thế bài thực hành: đánh dấu khi thêm, giữ cha lần đầu.
7     validate(graph, start, goal, h)
8     heap = [(h[start], start)]
9     discovered, parent = {start}, {start: None}
10    order, trace = [], []
11    expanded, peak = 0, 1
12    view = lambda: [f"{u}(h={p})" for p, u in sorted(heap)]
13    record(trace, "INIT", start, [], view(), step=step)
14    while heap:
15        before = view()
16        priority, u = heappop(heap)
17        order.append(u)
18        if u == goal:
19            record(trace, "GOAL", u, before, view(), step=step)
20            return make_result(graph, parent, goal, order, expanded, 0,
21                              0, peak, trace, True)
22        # B2. Giữ các ứng viên cũ trong heap, thêm hàng xóm chưa phát hiện.
23        for v, weight in graph[u]:
24            if v not in discovered:
25                discovered.add(v)
26                parent[v] = u
27                heappush(heap, (h[v], v))
28        expanded += 1
29        peak = max(peak, len(heap))
30        record(trace, "EXPAND", u, before, view(), step=step)
31    record(trace, "FAIL", goal, [], [], step=step)
32    return make_result(graph, parent, goal, order, expanded, 0,
33                      0, peak, trace, False)

```

GREEDY: chạy và đối chiếu ví dụ chung

```
python greedy_demo.py --step
python greedy_demo.py --summary
```

Bước	Xử lý	Biên sau bước; phần tử lấy tiếp ở bên trái
0	Khởi tạo S	$[S_4]$
1	Lấy S	$[G_0, A_3, B_4]$
2	Đích G	$[A_3, B_4]$

Đường $S - G$, giá 10; order S, G , expanded 1, skipped 0, peak 3.

Trong ví dụ chung, có ngay G trong biên với $h=0$, nên Greedy lấy G và trả đường đắt giá 10. Điều này không có nghĩa mọi ví dụ Greedy đều chỉ có hai bước; nó bộc lộ việc thuật toán không dùng chi phí quá khứ để ưu tiên.

Ví dụ thứ hai: không quên các ứng viên cũ

```
python run_demo.py --algo greedy --case frontier --step
```

Đỉnh	Danh sách cạnh ra (v, c)	h
S	$(A, 1), (B, 2)$	4
A	$(D, 2)$	1
B	$(G, 3)$	2
D	$(G, 7)$	6
G	$\emptyset$	0

Bước	Xử lý	Biên sau bước; phần tử lấy tiếp ở bên trái
0	Khởi tạo S	$[S_4]$
1	Lấy S	$[A_1, B_2]$
2	Lấy A	$[B_2, D_6]$
3	Lấy B	$[G_0, D_6]$
4	Đích G	$[D_6]$

Sau A , heap còn $B(h=2)$ từ bước trước và $D(h=6)$ mới sinh, nên chọn B . Đường $S-B-G$ có giá 5. Các câu lệnh `heappush` giữ mọi ứng viên, khác với cách sai `current = min(graph[current], ...)`.

9. Cài đặt A*

Ý tưởng trước khi đọc mã

Kế thừa cơ chế cập nhật giá của UCS nhưng dùng $f=g+h$ để ưu tiên. bản ghi chứa $(f, \text{tên đỉnh}, g)$, để hòa f theo tên và vẫn kiểm được bản ghi cũ bằng g. Không lấy h thay cho g; không cộng h vào chi phí thật của đường.

Hai lựa chọn cho CLOSED

Slide trình bày đóng-một-lần và bảo đảm khi h nhất quán. Mã dưới đây bổ sung `reopen=True`: nếu có g rẻ hơn vào đỉnh đã đóng, bỏ đỉnh khỏi closed, sửa cha, đưa bản ghi mới vào heap. Với `reopen=False`, bỏ ứng viên vào closed *trước mọi cập nhật*. Phần mở lại được dùng để thí nghiệm với h chấp nhận được nhưng không nhất quán.

Mã A* hoàn chỉnh trong astar_demo.py

```

1 from heapq import heappush, heappop
2 from math import inf
3 from common import validate, record, make_result
4
5 def astar(graph: dict, start: str, goal: str, h: dict,
6           step: bool = False, reopen: bool = True) -> dict:
7     # B1. Tuple (f, ten đỉnh, g): cung f thì theo ten; g dùng nhận phiếu cu.
8     validate(graph, start, goal, h)
9     heap = [(h[start], start, 0)]
10    best_g, parent = {start: 0}, {start: None}
11    closed, order, trace = set(), [], []
12    expanded = skipped = reopened = 0
13    peak = 1
14    view = lambda: [f"{u}(g={g},f={f})" for f, u, g in sorted(heap)]
15    record(trace, "INIT", start, [], view(), step=step)
16    while heap:
17        before = view()
18        f, u, g = heappop(heap)
19        # B2. Chỉ xử lý bản ghi mang chi phí tốt nhất đang biết.
20        if g != best_g[u] or u in closed:
21            skipped += 1
22            record(trace, "SKIP", u, before, view(), "old/closed", step)
23            continue
24        order.append(u)
25        if u == goal:
26            record(trace, "GOAL", u, before, view(), f"g={g}", step)
27            return make_result(graph, parent, goal, order, expanded, skipped,
28                              reopened, peak, trace, True)
29        closed.add(u)
30        changes = []
31        for v, weight in graph[u]:
32            new_g = g + weight
33            # B3. Bản đóng-mot-lan bỏ luôn ứng viên vào đỉnh đã đóng.
34            if not reopen and v in closed:
35                continue
36            if new_g < best_g.get(v, inf):
37                best_g[v] = new_g
38                parent[v] = u
39                # B4. Mò lại: loại v khỏi closed trước khi đưa bản mới vào heap.
40                if v in closed:
41                    closed.remove(v)
42                    reopened += 1
43                    changes.append(f"REOPEN {v}")
44                heappush(heap, (new_g + h[v], v, new_g))
45                changes.append(f"{v}:g={new_g},f={new_g+h[v]},parent={u}")
46        expanded += 1
47        peak = max(peak, len(heap))
48        record(trace, "EXPAND", u, before, view(), "; ".join(changes), step)
49    record(trace, "FAIL", goal, [], [], step=step)
50    return make_result(graph, parent, goal, order, expanded, skipped,
51                      reopened, peak, trace, False)

```

A*: chạy và đối chiếu ví dụ chung

```
python astar_demo.py --step
python astar_demo.py --summary
```

Bước	Xử lý	Biên sau bước; phần tử lấy tiếp ở bên trái
0	Khởi tạo S	$[S_{0;4}]$
1	Lấy S	$[A_{2;5}, B_{1;5}, G_{10;10}]$
2	Lấy A	$[B_{1;5}, C_{4;6}, G_{10;10}]$
3	Lấy B	$[C_{4;6}, D_{3;6}, G_{10;10}]$
4	Lấy C	$[D_{3;6}, G_{6;6}, G_{10;10}]$
5	Lấy D	$[G_{6;6}, G_{10;10}]$
6	Đích G	$[G_{10;10}]$

Đường $S - A - C - G$, giá 6; order S, A, B, C, D, G , expanded 5, skipped 0, peak 3.

Sau S: A(g=2,h=3,f=5), B(g=1,h=4,f=5), G(g=10,f=10). A và B hòa f nhưng A theo tên đứng trước. A tạo C(g=4,f=6); B có f=5 nên vẫn được xét trước C. Sau đó C và D cùng f=6, C trước D. Goal có f=6 nhưng D cũng f=6 và tên D trước G nên còn mở rộng D. “h tốt” không có nghĩa chỉ mở rộng đường lời giải.

Thí nghiệm mở lại, dữ liệu mẫu khác Bài 5

```
python run_demo.py --algo astar --case reopen --no-reopen
python run_demo.py --algo astar --case reopen --step
```

Đỉnh	Danh sách cạnh ra (v, c)	h
S	$(A, 2), (B, 1)$	2
A	$(C, 1)$	5
B	$(C, 4)$	1
C	$(G, 4)$	1
G	$\emptyset$	0

Bảng h chấp nhận được; chỉ cạnh A–C vi phạm: $5 > 1 + 1$. Bản không mở lại trả S–B–C–G giá 9. Bản mở lại cải thiện C từ g=5 xuống 3, trả S–A–C–G giá 7; C được mở rộng hai lần. Hãy đặt breakpoint ở `closed.remove(v)` để nhìn tận nơi việc mở lại xảy ra.

Cài đặt mở lại không cứu được mọi heuristic ước lượng quá. Muốn bảo đảm tối ưu khi dùng h chấp nhận được trong môi trường hữu hạn/trọng số dương của bộ, cần giữ đúng cập nhật, bỏ bản ghi cũ và thời điểm dừng. Validate không tự suy ra tính chấp nhận được.

10. So sánh năm chiến lược, không trộn mục tiêu

Thuật toán	Chọn gì?	Đường mẫu	Giá	Expanded
DFS	LIFO	$S - A - C - G$	6	3
BFS	FIFO	$S - G$	10	3
UCS	g nhỏ nhất	$S - A - C - G$	6	5
GREEDY	h nhỏ nhất	$S - G$	10	1
A*	$g + h$ nhỏ nhất	$S - A - C - G$	6	5

Trên ví dụ này DFS cũng tìm giá 6, nhưng đó không phải bảo đảm tối ưu của DFS. Greedy mở rộng ít hơn nhưng đường đắt. A* và UCS mở rộng cùng số lần ở ví dụ chung; không nên viết “A* luôn ít hơn UCS” dựa vào trực giác hình ảnh.

Mê cung có nước

```
python run_demo.py --algo bfs --case maze --summary
python run_demo.py --algo ucs --case maze --summary
python run_demo.py --algo greedy --case maze --summary
python run_demo.py --algo astar --case maze --summary
```

Bộ chuyển mê cung trong `common.py` duyệt từng ô, tạo các cạnh ngang/dọc và gán phí ô đi vào. Tọa độ được mã hóa thành chuỗi “r,c”. Với mã tên chuỗi, hòa ưu tiên dùng từ điển chuỗi; không ngầm chuyển thành so số. Dữ liệu nhỏ trong bộ có chỉ số một chữ số.

Manhattan tới đích là $|r - r_G| + |c - c_G|$. Khi đổi goal, phải tính lại bảng h ; không dùng h của đích cũ. Bức tường làm mất cạnh, không làm ô đó thành đỉnh có phí rất lớn. Nhớ giữ chữ S/G khi tô đường bằng dấu *.

Vì sao phần helper được tách?

`maze_graph` giải quyết mô hình; năm hàm tìm kiếm giải quyết chiến lược; `draw_maze` giải quyết hiển thị. Tách ba việc giúp thử một thuật toán trên nhiều đồ thị và thử nhiều thuật toán trên cùng dữ liệu mà không viết lại từ đầu.

11. Dừng ở dòng mã thật trong VS Code

1. Mở `structures_demo.py`; bấm lên trái tại dòng `popped = stack.pop()` để có chấm đỏ.
2. Mở Run and Debug (Ctrl+Shift+D), chọn cấu hình “File đang mở – dùng breakpoint và F10” đã có trong `.vscode/launch.json`, nhấn F5. Nếu không dùng cấu hình, chọn Python Debugger: Debug Python File.
3. Khi dừng, dòng vàng là dòng sắp thực hiện. Xem Variables/Locals hoặc đưa chuột lên biến. Nhấn F10 để thực hiện dòng hiện tại. Stack đổi từ `['B','A']` sang `['B']`, `popped='A'`; `before` vẫn là `['B','A']`.
4. Với `astar_demo.py`, đặt breakpoint tại `f, u, g = heappop(heap)`. Xem `heap`, `best_g`, `parent`, `closed`. F10 bước qua dòng; F11 đi vào hàm; F5 chạy tới breakpoint kế; Shift+F5 dừng.

Cấu hình dùng console: `integratedTerminal`, vì `input()` cần Terminal nhận Enter. Để quan sát mở lại, chạy cấu hình/tập với tham số `-case reopen`; nếu chạy ví dụ có h nhất quán thì breakpoint ở `closed.remove(v)` có thể không bao giờ dừng – đó là đúng với dữ liệu, không nhất thiết lỗi trình gỡ lỗi.

Khi được gọi để chữa bài trên lớp

Chuẩn bị ba dự đoán: (1) phần tử sắp lấy, (2) chi phí mới, (3) cha nào đổi. Sau mỗi F10, nói bằng lời điều gì thay đổi và dòng nào làm thay đổi. Không cần đọc mọi dòng import/helper; tập trung các quyết định của thuật toán.

Bài tập 2 phút về before và after

Đặt breakpoint trước `pop`. Ghi hai biến `before`, `stack`; bước qua `pop` rồi ghi lại. Giải thích vì sao `before = stack.copy()` khác `before = stack`. Bài này kiểm tra một lỗi hiển thị thực tế có thể làm người học hiểu sai thuật toán dù bản thân `pop` hoàn toàn đúng.

12. Kiểm thử từ nhỏ đến lớn

Mức 1: đường một cạnh. Dễ tính tay, xác nhận goal và path. **Mức 2: đầu trùng đích/không có đường/chu trình.** Xác nhận dừng. **Mức 3: hai đường cùng tới một đỉnh.** Xác nhận cập nhật/cha. **Mức 4: đồng hạng và bản ghi cũ.** Xác nhận quy ước. **Mức 5: đối chiếu độc lập.** Liệt kê đường đơn trên đồ thị rất nhỏ để kiểm giá tối ưu.

```
from astar_demo import astar
from ucs_demo import ucs

g = {"S": [("G", 10), ("A", 1)],
      "A": [("G", 2)], "G": [], "X": []}
h = {"S": 2, "A": 1, "G": 0, "X": 0}
a = astar(g, "S", "G", h)
assert a["path"] == ["S", "A", "G"]
assert a["cost"] == 3
assert a["expanded"] == 2
assert astar(g, "G", "G", h)["cost"] == 0
assert astar(g, "S", "X", h)["path"] is None
zero = {u: 0 for u in g}
assert astar(g, "S", "G", zero)["order"] == ucs(g, "S", "G")["order"]
```

Các assert này kiểm mã mẫu. Khi viết `test_bai_lam.py`, phải import `bai_lam`, không import `demo`; nếu không sẽ vô tình kiểm bài của người khác thay cho mã mình vừa viết.

```
python -m unittest -v test_mau
python -m unittest -v test_cong_khai.PublicTests.test_dfs
python -m unittest -v test_bai_lam
```

`test_mau` có sẵn để kiểm chương trình học mẫu. `test_cong_khai` có sẵn để thử bài nộp trên dữ liệu nhỏ. `test_bai_lam.py` do sinh viên tự tạo, nên lệnh cuối chỉ chạy được sau khi đã tạo tệp này. Test thất bại cần đọc *giá trị kỳ vọng* và *giá trị thực*, không chỉ nhìn chữ FAIL.

Kiểm tra không sửa đầu vào

```
from copy import deepcopy
before = deepcopy(g)
astar(g, "S", "G", h)
assert g == before
```

Đổi dữ liệu để làm thí nghiệm trên bản sao, không sửa âm thầm dữ liệu chuẩn trong một hàm tìm kiếm. Một hàm đúng phải dùng lại được cho đồ thị khác.

13. Từ mã mẫu sang năm bài phải nộp

1. Làm xong phần mô phỏng tay liên quan. Mở đúng ký hiệu hàm trong `bai_lam.py`; viết lại khung từ trí nhớ và dùng mã mẫu để đối chiếu phần chưa chắc.
2. Cài đặt đúng một thuật toán. Được gọi helper `common.py`; không gọi hàm `demo` thay cho phần phải cài đặt. Tên biến có thể khác, quy tắc và hợp đồng phải giữ.
3. Kiểm thử trên đồ thị 2-3 đỉnh trước; chạy test công khai của riêng hàm. Chưa cài các hàm khác không cản việc chạy một bài.
4. Chạy dữ liệu bài nộp bằng lệnh dưới, rồi mở output để đối chiếu `path/cost/order` với dự đoán. Runner chỉ chạy *dữ liệu chính*; các ca biến đổi và lỗi đầu vào vẫn do sinh viên tự kiểm thử.

```
python chay_bai_lam.py --bai 7
python chay_bai_lam.py --bai 8
python chay_bai_lam.py --bai 9
python chay_bai_lam.py --bai 10
python chay_bai_lam.py --bai 11 --step
```

Mẫu một đoạn báo cáo

“Tôi dự đoán B được lấy trước A vì $g(B)=1$ nhỏ hơn $g(A)=2$. B tạo đường tới C giá 6; khi mở rộng A, đường đến C chỉ có giá 4 nên đổi cha C từ B sang A. bản ghi C(6) vẫn ở heap; sau này bị bỏ vì không khớp `best_g[C]`. Tôi kiểm chứng bằng một `assert` cho đường và một `assert` cho số bản ghi bị bỏ.”

Không chỉ chép đoạn này; viết đoạn tương ứng với dữ liệu và mã của mình. Một bảng chỉ có kết quả không chứng minh sinh viên biết cơ chế. Cần chỉ ra được dòng nào thực hiện quyết định đã mô tả.

Tự kiểm trước khi nộp

Chạy lại từ một thư mục mới sau khi giải nén. Kiểm tra không cần đường dẫn máy cá nhân; không cần thư mục `teacher`; không còn `import` nhằm `demo`; báo cáo ghi đúng phiên bản dữ liệu; đường trả về thật sự gồm các cạnh hợp lệ; tổng chi phí cộng lại đúng; các số `expanded/skipped` thống nhất với `trace`.

14. Các lỗi thường gặp và cách xác định

Biểu hiện	Nguyên nhân cần kiểm tra	Cách xử lý
can't open file	Terminal ở sai thư mục hoặc tên sai.	Gõ dir; phải thấy run_demo.py. Open Folder đúng gốc; không cd python trong gói này.
ModuleNotFoundError: common	Chỉ tải một tệp demo.	Giải nén toàn bộ; giữ helper cạnh demo.
Hiện dấu >>	Đang ở Python REPL, không phải shell.	Gõ exit(), rồi gõ lệnh python ... trong Terminal.
Không chờ Enter	Quên -step hoặc chọn chế độ summary.	Dùng riêng -step; hai cờ bị chương trình từ chối nếu cùng dùng.
Enter không tác dụng	Đang bấm ở editor/Output.	Bấm Terminal tích hợp; không dùng Code Runner Output cho input().
NotImplementedError	Hàm trong bai_lam.py còn khung.	Hoàn thiện hàm hoặc chạy demo thay vì runner bài nộp.
Kết quả đúng, trace sai	Sai quy tắc hòa/đánh dấu/kiểm tra đích.	Đối chiếu từng bước đầu tiên bị khác; không chỉ sửa để khớp đáp án cuối.
Stack before không có phần tử pop	Lưu alias hoặc pop trong đối số print.	Chụp copy trước pop, rồi print ở câu lệnh sau.

Tài liệu tham chiếu

Bài giảng của học phần: Nguyễn Bích Vân, Bài 2 – Tìm kiếm mù và Bài 3 – Tìm kiếm dựa trên kinh nghiệm; phạm vi áp dụng như mô tả đầu hướng dẫn.

API và thao tác lập trình bổ sung:

Python: <https://docs.python.org/3/tutorial/datastructures.html>

Heap: <https://docs.python.org/3/library/heapq.html>

Kiểm thử: <https://docs.python.org/3/library/unittest.html>

VS Code: <https://code.visualstudio.com/docs/python/debugging>

Tài liệu nền bổ sung về tìm kiếm có thông tin: <https://inst.eecs.berkeley.edu/~cs188/textbook/search/informed.html>